

FAST

FORMAÇÃO ACCELERADA EM
SOLUÇÕES DE TECHDESIGN



C.E.S.A.R.

Automação de Testes Web

Todos os direitos reservados.

Este material, ou qualquer parte dele, não pode ser reproduzido,
divulgado ou usado de forma alguma sem autorização escrita.



Testes de Automação Web

Automação Web
Aula 2



Estrutura do curso

Conteúdo:

Aula 1:

- Fundamentos da Automação de Testes
- Diferenças entre testes manuais e automatizados
- Estrutura de um Script de Teste
- Ferramentas

Aula 2:

- Identificação de Elementos
- Elementos dinâmicos e boas práticas
- Introdução ao padrão page object

Aula 3:

- Organização com page object
- Comandos, esperas e validações
- Comandos, esperas e interações básicas

Aula 4:

- Estrutura de projeto e boas práticas
- Modularização e reuso de código
- Construção de suite de testes

Aula 5:

- Criação de fluxo de teste completo
- Prática guiada
- Bonus: CI/CD + Paralelização

Intervalo: 15 à 20 minutos

Material das aulas: NEO

Avaliação Final: 5 questões objetivas

O que veremos hoje?

- Identificação de Elementos na Interface Web
- Elementos dinâmicos e boas práticas.
- Introdução ao padrão Page Object (POM)

Identificação de Elementos

O que são elementos web?

Elementos web são todos os componentes que compõem uma página HTML. Exemplos:

- `<input>` : campo de texto
- `<button>` : botão
- `<a>` : link
- `<div>`, `<span>`, etc.

Para automatizar testes, precisamos **identificar e localizar** esses elementos de forma precisa e estável.

Identificação de Elementos

Como inspecionar elementos?

A inspeção geralmente é feita usando o **DevTools** do navegador (como o Google Chrome ou Firefox).

Passo a passo no Chrome:

1. Clique com o botão direito sobre o elemento desejado.
2. Escolha a opção "Inspecionar".
3. O navegador abrirá o painel do DevTools, destacando o HTML correspondente.
4. No painel, você pode ver os **atributos** do elemento, como:
 - `id`
 - `class`
 - `name`
 - `type`
 - `placeholder`
 - `value`

Identificação de Elementos

Localizadores comuns

Os localizadores são as formas de "dizer" ao seu código qual elemento você quer manipular. Os mais usados são:

Localizador	Exemplo	Observação
<code>id</code>	<code>driver.findElement(By.id("login"))</code>	Geralmente único, preferido se disponível
<code>name</code>	<code>By.name("username")</code>	Útil para formulários
<code>className</code>	<code>By.className("btn-primary")</code>	Pode haver múltiplos com a mesma classe
<code>tagName</code>	<code>By.tagName("button")</code>	Muito genérico
<code>linkText</code>	<code>By.linkText("Clique aqui")</code>	Para links
<code>cssSelector</code>	<code>By.cssSelector(".menu > li > a")</code>	Poderoso e flexível
<code>xpath</code>	<code>By.xpath("//*[@type='text']")</code>	Muito usado, mas pode ser sensível a mudanças

Identificação de Elementos

Exemplo de HTML:

html

[Copy](#) [Edit](#)

```
<form id="login-form">  
  <label for="email">E-mail:</label>  
  <input type="email" id="email" name="userEmail" class="input-field" placeholder="Digi<br/></form>
```

Elemento-alvo:

Queremos mapear o elemento:

html

[Copy](#) [Edit](#)

```
<input type="email" id="email" name="userEmail" class="input-field" />
```


Identificação de Elementos

✓ Mapeando por CSS Selector:

1. Por `id`:

CSS

[Copy](#) [Edit](#)

```
#email
```

2. Por `class`:

CSS

[Copy](#) [Edit](#)

```
.input-field
```

3. Por `name`:

CSS

[Copy](#) [Edit](#)

```
input[name='userEmail']
```

4. Por estrutura:

CSS

[Copy](#) [Edit](#)

```
form#login-form input.input-field
```

Identificação de Elementos

✓ Mapeando por XPath:

1. Por `id`:

xpath

Copy Edit

```
//input[@id='email']
```

2. Por `class`:

xpath

Copy Edit

```
//input[@class='input-field']
```

3. Por `name`:

xpath

Copy Edit

```
//input[@name='userEmail']
```

4. Por estrutura (descendente do form):

xpath

Copy Edit

```
//form[@id='login-form']//input[@type='email']
```

Identificação de Elementos

Comparando os dois:

Critério	CSS Selector	XPath
Mais legível	✓ Sim	✗ Pode ser mais verboso
Suporte a caminho reverso (<code>parent</code>)	✗ Não	✓ Sim (<code>..</code> ou <code>parent::</code>)
Suporte nativo no navegador	✓ Sim (<code>querySelector</code>)	✓ Sim (<code>\$x()</code> no console)
Mais rápido (geralmente)	✓ Sim	✗ Pode ser ligeiramente mais lento
Mais expressivo	✗ Limitado	✓ Muito mais poderoso e flexível

Identificação de Elementos

Exemplo em Selenium (Java):

java

 Copy  Edit

```
// CSS Selector
WebElement campoEmailCss = driver.findElement(By.cssSelector("input[name='userEmail']"))

// XPath
WebElement campoEmailXPath = driver.findElement(By.xpath("//input[@name='userEmail']"))
```

**Quando o elemento é
dinâmico, o que eu faço ?**

Elementos dinâmicos e boas práticas

Exemplos de elementos dinâmicos

html

 Copy  Edit

```
<input id="user_84372" class="input-xyz-123" name="username">
```

- O `id="user_84372"` pode mudar para `user_93212` no próximo acesso.
- A `class="input-xyz-123"` pode ser gerada aleatoriamente.

Elementos dinâmicos e boas práticas

Tabela de Seletores CSS - Principais Opções

Tipo de Seletor	Sintaxe CSS	Exemplo	O que faz?
Por ID	<code>#id</code>	<code>#email</code>	Seleciona elemento com <code>id="email"</code>
Por classe	<code>.classe</code>	<code>.input-field</code>	Seleciona elemento com <code>class="input-field"</code>
Por tag	<code>tag</code>	<code>input</code> , <code>button</code>	Seleciona todos os elementos dessa tag
Por atributo (valor exato)	<code>[atributo="valor"]</code>	<code>input[name="email"]</code>	Seleciona elementos com atributo igual ao valor
Atributo contém valor	<code>[atributo~="valor"]</code>	<code>input[placeholder~="e-mail"]</code>	Seleciona se o atributo contém o valor
Atributo começa com	<code>[atributo^="valor"]</code>	<code>input[name^="user"]</code>	Seleciona se o atributo começa com o valor
Atributo termina com	<code>[atributo\$="valor"]</code>	<code>input[name\$="Email"]</code>	Seleciona se o atributo termina com o valor
Combinação de seletores	<code>tag.classe#id[atributo="valor"]</code>	<code>input.input-field#email[name="userEmail"]</code>	Usa múltiplos critérios no mesmo seletor
Elemento dentro de outro	<code>pai filho</code>	<code>form input</code>	Seleciona todos os <code>input</code> dentro de um <code>form</code> (em qualquer nível)

Elementos dinâmicos e boas práticas

✓ Cenário: Tela de Login com Elementos Dinâmicos

HTML Simulado

```
html                                                                    Copy Edit

<form id="login-form">
  <div class="input-wrapper">
    <label for="user-input">Usuário</label>
    <input id="input_92837" name="username" type="text" class="fld-abc-9381" placeholder="Digite seu usuário" />
  </div>
  <div class="input-wrapper">
    <label for="pass-input">Senha</label>
    <input id="input_47362" name="password" type="password" class="fld-def-8237" placeholder="Digite sua senha" />
  </div>
  <button type="submit" class="btn-submit">Entrar</button>
</form>
```

 Note que os `id` e `class` dos `<input>` são dinâmicos (`input_92837`, `fld-abc-9381` etc.), ou seja, mudam a cada carregamento.

Elementos dinâmicos e boas práticas

```
<form id="login-form">
  <div class="input-wrapper">
    <label for="user-input">Usuário</label>
    <input id="input_92837" name="username" type="text" class="fld-abc-9381"
placeholder="Digite seu usuário">
  </div>
  <div class="input-wrapper">
    <label for="pass-input">Senha</label>
    <input id="input_47362" name="password" type="password" class="fld-def-8237"
placeholder="Digite sua senha">
  </div>
  <button type="submit" class="btn-submit">Entrar</button>
</form>
```

Elementos dinâmicos e boas práticas

CSS Selectors

Campo de usuário:

```
CSS
input[name="username"]
```

Ou, com base na estrutura:

```
CSS
form#login-form input[placeholder="Digite seu usuário"]
```

Campo de senha:

```
CSS
input[placeholder$="senha"]
```

XPath

Campo de usuário (por `name`):

```
xpath
//input[@name='username']
```

Campo de usuário (por texto do label):

```
xpath
//label[text()='Usuário']/following-sibling::input
```

Campo de senha (por placeholder):

```
xpath
//input[contains(@placeholder, 'senha')]
```

Elementos dinâmicos e boas práticas

Boas práticas na identificação de elementos

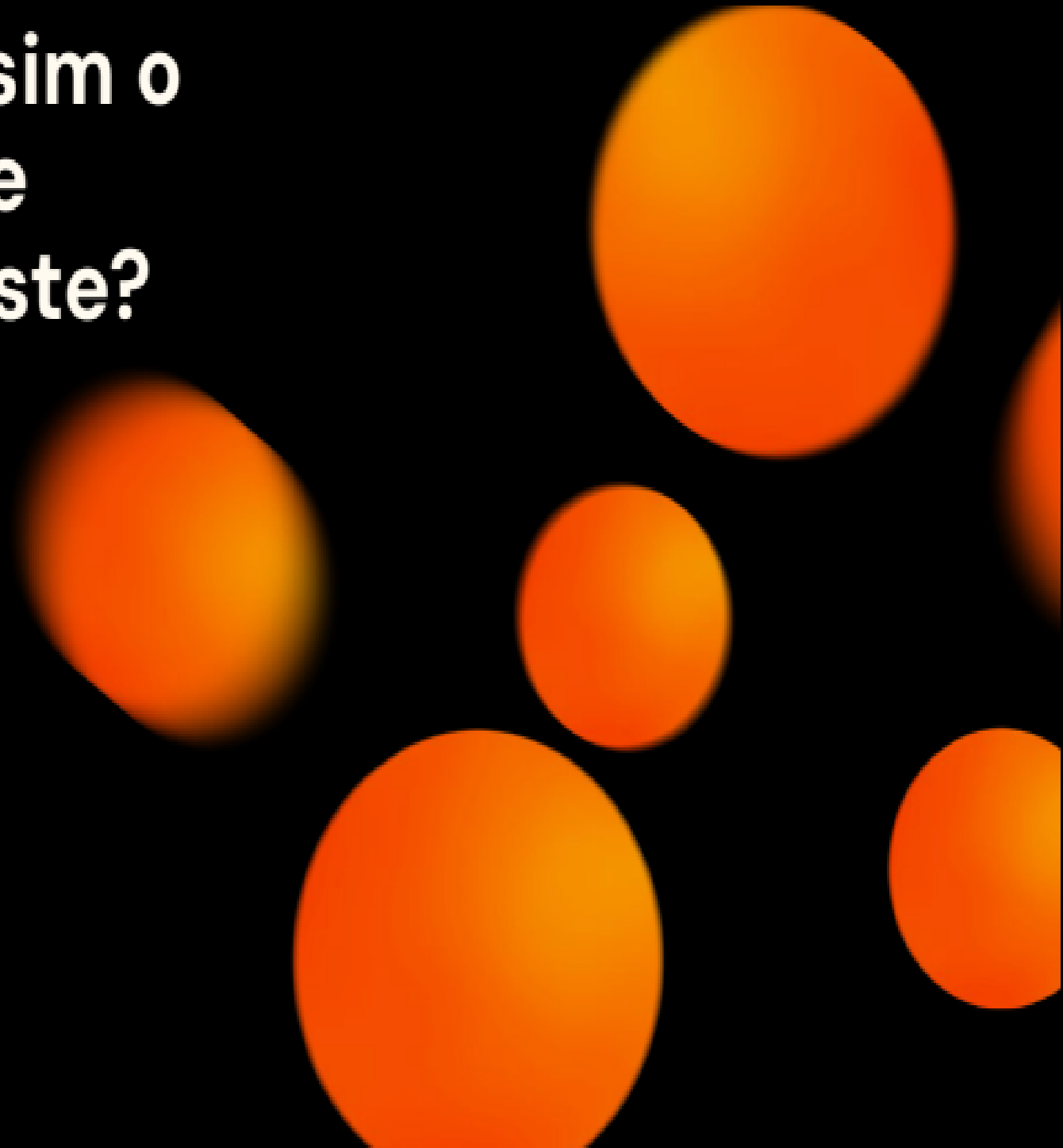
- Prefira `id` quando disponível — é o mais confiável.
- Use seletores que **não dependam da posição visual** (como `nth-child()`).
- Evite usar `class` se ela mudar dinamicamente ou for muito genérica.
- Use **XPath** ou **CSS Selectors** para cenários complexos ou ausência de identificadores únicos.
- Mantenha os localizadores **centralizados** em uma camada do framework, para facilitar a manutenção (ex: Page Object Model).

INTERVALO!



20 min.

**E se mesmo assim o
elemento mudar e
quebrar o meu teste?**



Introdução ao padrão Page Object (POM)

Estrutura do Projeto (exemplo simples)

CSS

```
src/  
├── pages/  
│   └── LoginPage.java  
├── tests/  
│   └── LoginTest.java  
└── utils/  
    └── DriverFactory.java
```

Introdução ao padrão Page Object (POM)

1. Classe LoginPage.java

```
java Copy Edit  
  
package pages;  
  
import org.openqa.selenium.By;  
import org.openqa.selenium.WebDriver;  
import org.openqa.selenium.WebElement;  
  
public class LoginPage {  
    private WebDriver driver;  
  
    // Construtor  
    public LoginPage(WebDriver driver) {  
        this.driver = driver;  
    }  
  
    // Elementos identificados de forma resistente (dinâmicos)  
    private By inputUsername = By.cssSelector("input[name='username']");  
    private By inputPassword = By.xpath("//input[contains(@placeholder, 'senha')]");  
    private By botaoEntrar = By.cssSelector("button.btn-submit");  
  
    // Ações da página  
    public void preencherUsuario(String usuario) {  
        driver.findElement(inputUsername).sendKeys(usuario);  
    }  
  
    public void preencherSenha(String senha) {  
        driver.findElement(inputPassword).sendKeys(senha);  
    }  
  
    public void clicarEntrar() {  
        driver.findElement(botaoEntrar).click();  
    }  
}
```

Introdução ao padrão Page Object (POM)

2. Classe de Teste LoginTest.java

```
java Copy Edit

package tests;

import org.junit.After;
import org.junit.Before;
import org.junit.Test;
import org.openqa.selenium.WebDriver;

import pages.LoginPage;
import utils.DriverFactory;

public class LoginTest {
    private WebDriver driver;
    private LoginPage loginPage;

    @Before
    public void setup() {
        driver = DriverFactory.createDriver();
        driver.get("https://exemplo.com/login");
        loginPage = new LoginPage(driver);
    }

    @Test
    public void testLoginComSucesso() {
        loginPage.preencherUsuario("meu_usuario");
        loginPage.preencherSenha("minha_senha");
        loginPage.clicarEntrar();
        // Aqui você poderia adicionar assertivas de validação
    }

    @After
    public void tearDown() {
        driver.quit();
    }
}
```


Introdução ao padrão Page Object (POM)

Dicas:

- Os seletores usados no `LoginPage` são resistentes a mudanças dinâmicas de `id` e `class`.
- No POM, o foco é **abstrair os elementos e ações da página**. Isso melhora a manutenção e reutilização.
- Você pode evoluir o projeto com:
 - PageFactory (com `@FindBy`)
 - Frameworks de teste como JUnit5, TestNG, Cucumber
 - Suporte a múltiplas páginas, validações, waits, etc.

Introdução ao padrão Page Object (POM)

✓ Vantagens de usar Page Object Model (POM)

Vantagem	Explicação
1. Separação de responsabilidades	A lógica de navegação e interação com elementos fica separada da lógica dos testes. Isso deixa o código mais limpo, organizado e fácil de entender.
2. Reutilização de código	Um mesmo método pode ser chamado em vários testes. Por exemplo: <code>loginPage.preencherUsuario()</code> pode ser usado em múltiplos cenários.
3. Facilidade de manutenção	Se um seletor mudar (por exemplo, o <code>id</code> de um campo), você só precisa atualizar isso em um único lugar, dentro da Page.
4. Redução de duplicação	Evita escrever os mesmos comandos de <code>findElement</code> , <code>click</code> ou <code>sendKeys</code> em todos os testes.
5. Código mais legível	Os testes ficam mais descritivos e fáceis de entender, como se fossem "roteiros de ações". Exemplo: <code>loginPage.clicarEntrar()</code> em vez de <code>driver.findElement(...).click()</code> .
6. Padronização	Ajuda a manter uma estrutura de automação consistente, com nomenclatura e organização comuns para todas as páginas.
7. Integração com outros padrões	Facilita o uso combinado com padrões como o Factory , Data-Driven , Cucumber/Gherkin , entre outros.
8. Suporte à escalabilidade	Ideal para aplicações com muitas páginas e testes. Você consegue escalar seu projeto sem virar um caos.
9. Melhora a depuração	Se um teste falha, você sabe que o problema pode estar na Page ou no Teste — isso facilita o rastreamento de erros.
10. Facilita a adoção por outros membros do time	QA, Devs ou novos membros entendem mais facilmente como o projeto está estruturado.

**DÚVIDAS?!
:)**





C . E . S . A . R

MUITO OBRIGADO!

© CESAR | Todos os direitos reservados